

# SO-101 Training Framework & Orchestrator Architecture

Phang Hong Hao (潘洪浩)

Tsinghua University

2026 年 5 月 13 日

# Outline

---

Training Framework Overview

Speed Benchmark

Optimal Configuration

Reference Models

Orchestrator Architecture

# Training Framework Overview

本项目在 RTX 6000D 上同时训练两种 Policy:

|                   | PushT Diffusion            | SO-101 ACT                  |
|-------------------|----------------------------|-----------------------------|
| <b>Policy</b>     | Diffusion (UNet)           | ACT (Transformer)           |
| <b>Params</b>     | 263M                       | 84M                         |
| <b>Dataset</b>    | 1erobot/pusht (26K frames) | so101_push_sim (30K frames) |
| <b>Task</b>       | 2D Push-T (验证)             | SO-101 机械臂推物                |
| <b>batch_size</b> | 64                         | 128                         |
| <b>steps</b>      | 100K                       | 50K                         |
| <b>Device</b>     | cuda:7                     | cuda:6                      |

# Architecture Difference

## Architecture Comparison: Diffusion vs ACT

### Puhti Diffusion

UNet 263M params

Conditional denoising

chunk\_size = 1

→ Lightweight compute / step

Speed  
~13 step/s

### SO-101 ACT

Transformer Encoder 4L

+ CVAE Decoder 7L

chunk\_size = 100 ( $O(n^2)$ )

→ Heavy compute / step

→ batch\_size cap at 128

~1.9 step/s

VS

Root cause: ACT chunk\_size=100 Transformer self-attention  $O(n^2)$  → 79GB VRAM per step at batch=128

## ACT 瓶颈

chunk\_size=100 的 Transformer decoder 自注意力复杂度  $O(n^2)$ , 128 batch 时单 step 显存占用 ~79GB (RTX 6000D 单卡上限), 无法进一步增大 batch。

## Speed Benchmark: PushT Diffusion

**PushT Diffusion** (263M params, batch=64)

| Metric         | pyav           | torchcodec   | 加速比  |
|----------------|----------------|--------------|------|
| Speed          | 0.5–0.7 step/s | 12–15 step/s | ~20x |
| 100K steps ETA | ~40h           | ~2h          |      |
| Loss           | —              | 0.022–0.024  |      |

### Key Finding

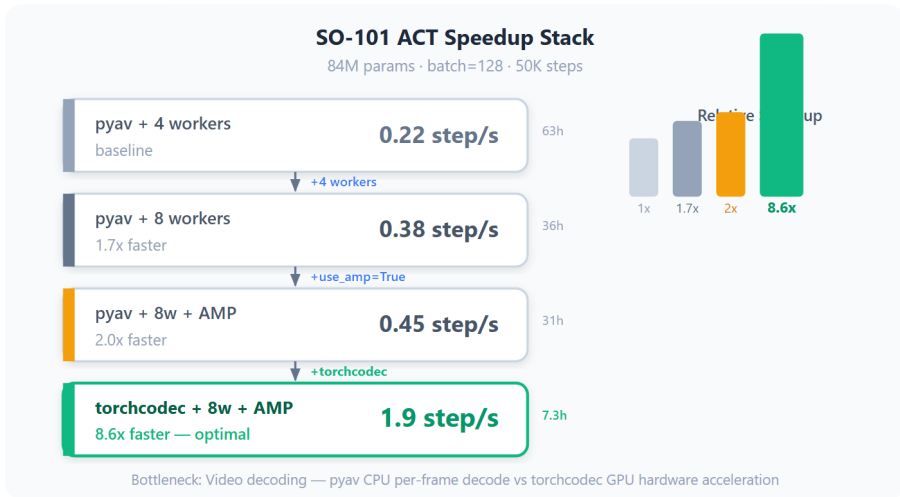
torchcodec 用 GPU 硬件加速视频解码，比 pyav CPU 逐帧解码快 20 倍。

# Speed Benchmark: SO-101 ACT

**SO-101 ACT** (84M params, batch=128)

| Config                         | Speed             | 50K ETA      | 备注               |
|--------------------------------|-------------------|--------------|------------------|
| pyav, 4 workers                | 0.22 step/s       | ~63h         | 最初配置             |
| pyav, 8 workers                | 0.38 step/s       | ~36h         | 增大 workers       |
| pyav, 8 workers, AMP           | 0.45 step/s       | ~31h         | 开启混合精度           |
| nvgreen!15 torchcodec, 8w, AMP | <b>1.9 step/s</b> | <b>~7.3h</b> | <b>最优 (8.6x)</b> |

# Speedup Stack



GPU 利用率始终 100%，但 data\_s (数据加载时间) 在 pyav 下远高于 torchcodec。

# Optimal Training Configuration

## Required Env Variables

```
# torchcodec libstdc++ fix
export LD_PRELOAD=~/.miniconda3/\
envs/lerobot/lib/libstdc++.so.6

# LeRobot v0.5.1 bug fix
export CUDA_VISIBLE_DEVICES=6

# HF mirror
export HF_ENDPOINT=\
  https://hf-mirror.com
```

## Launch Command

```
python -u -c "
import sys
sys.argv = [
    'lerobot-train',
    '--dataset.repo_id=PhangHongHao/so101_push_sim',
    '--dataset.video_backend=torchcodec', # 8.6x
    '--policy.type=act',
    '--policy.device=cuda', # not cuda:N!
    '--policy.chunk_size=100',
    '--policy.dim_model=512',
    '--policy.n_heads=8',
    '--policy.n_encoder_layers=4',
    '--policy.n_decoder_layers=7',
    '--policy.use_vae=True',
    '--policy.use_amp=True', # ~1.2x
    '--batch_size=128', # max
    '--steps=50000',
    '--num_workers=8',
]
from lerobot.scripts.lerobot_train import main
main()
"
```



# Known Bugs & Workarounds

| Bug                                             | Root Cause                                          | Fix                                                                             |
|-------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------|
| LeRobot v0.5.1 只接受"cuda"/"cpu"/"xpu"/"mps"      | CUDA_VISIBLE_DEVICES=6 +<br>--policy.device=cuda    | is_amp_available("cuda:6") ValueError<br>is_amp_available crash 被 torchcodec 捕获 |
| 同上                                              | torchcodec DecodingError<br>openvino 需要新版 libstdc++ | LD_PRELOAD=.../libstdc++.so.6                                                   |
| CXXABI_1.3.15 not found<br>torchcodec 需要 FFmpeg | conda install -c conda-forge<br>ffmpeg              | libavutil.so.X not found<br>ACT chunk_size=100 $O(n^2)$ 显存                      |
| 使用 batch_size=128                               | batch_size=512 OOM                                  |                                                                                 |

# Why batch\_size=128 is the Maximum



# HuggingFace Reference Models (Survey of 100+)

| #                                      | Model                                    | Policy            |                         |
|----------------------------------------|------------------------------------------|-------------------|-------------------------|
| Sa7411/smolvla_so101_pick<br>2         | SmolVLA<br>TakuyaHiraoka/act_so101_pick  | 393<br>ACT        | 87.66% 成功率, 数据分割        |
| AdityaRege/so101-pick-place-act<br>4   | ACT<br>edge-inference/smolvla-so101-pick | 121<br>SmolVLA    | HF 最多点赞                 |
| davidlinjiahao/lerobot_so101_base<br>6 | ACT<br>Vizuara/dreamzero-so101-lora      | 1<br>World Action | MuJoCo 仿真, 66.7%@50K, 最 |

# Key Takeaways

## 1. ACT 是 SO-101 主流策略

- ▶ HF 上 30+ 个 ACT 模型, SmolVLA 约 4-5 个

## 2. SmolVLA 成功率更高但数据需求不同

- ▶ Sa74ll 达 87.66%, 靠数据分割而非调参

## 3. 本项目 pipeline 与 davidlinjiahao 最相似

- ▶ 都是 MuJoCo 仿真 + ACT + SO-101

## 4. 视频解码是训练瓶颈

- ▶ torchcodec 比 pyav 快 8-20x, 是 RTX 6000D 上最关键的优化

# Orchestrator Overview



YAML-driven pipeline orchestrator — 自动化完整模仿学习 workflow:  
数据采集 → 训练 → 评估 → 对比



## 未找到文件

它可能已被移动、编辑或删除。

ERR\_FILE\_NOT\_FOUND

# Core Components —Event Loop & State Machine

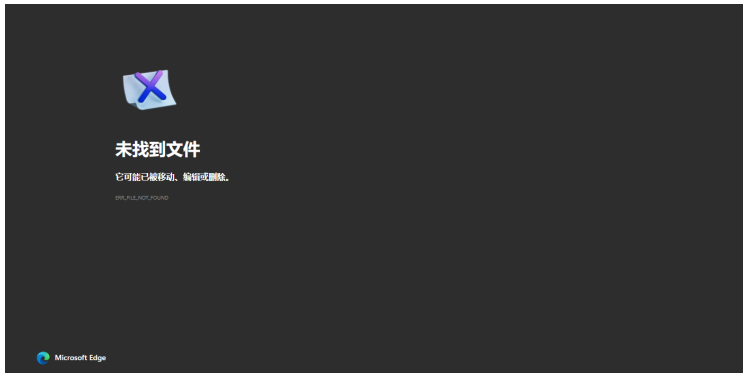


未找到文件

它可能已被移动、编辑或删除。

ERR\_FILE\_NOT\_FOUND

# Three-Layer Config Merge



Merge Priority (High → Low)

Phase-specific config > YAML defaults > Code defaults



# Training Plan YAML Format

```
plan_name: so101_push_plan
device: cuda:7

defaults:
    # Shared across all phases
    dataset: {repo_id: "PhangHongHao/so101_push_sim", fps: 30}
    training: {batch_size: 128, num_workers: 8}
    monitor: {poll_interval: 60}

phases:
- id: collect
  # Phase 1: Data Collection
  type: collection
  collection:
    trajectory_type: ik_push
    n_episodes: 300

- id: train_act
  # Phase 2: Training
  type: training
  depends_on: [collect]
  policy:
    type: act
    chunk_size: 100
    dim_model: 512
  training:
    max_steps: 50000

- id: eval_act
  # Phase 3: Evaluation
  type: evaluation
  depends_on: [train_act]
  eval:
    n_episodes: 20
    success_threshold: 0.05
```

# Usage

```
# Full pipeline (collection -> training -> evaluation)
python -m orchestrator_arm101.arm101_orchestrator \
    --plan training_plans/so101_push_plan.yaml \
    --device cuda:7 --fresh

# Dry run (print plan, no execution)
python -m orchestrator_arm101.arm101_orchestrator \
    --plan training_plans/so101_push_plan.yaml --dry-run

# Resume from specific phase (skip collection)
python -m orchestrator_arm101.arm101_orchestrator \
    --plan training_plans/rtx_train_plan.yaml \
    --start-from train_act --device cuda:6 --fresh
```

## Data Collection Pipeline

scene.xml → MjModel → scripted trajectory → render → LeRobotDataset → HF Hub

# Thank You

Questions?

Phang Hong Hao | Tsinghua University

 PhangHongHao